Website themes

Theme bundles

A theme is a directory. Three well-known filenames cover every output:

```
themes/my-theme/
  paged.typ         // Typst show rules for pdf/svg/png output
  document.html     // MiniJinja layout for a single-document HTML render
  site.html         // MiniJinja layout for website pages
  partials/
  styles/
  scripts/
```

Every file is optional. When a bundle is missing an entry file, that output kind falls back to the builtin default theme, together with the default's own partials, styles, and scripts. An empty `paged.typ` disables paged styling; an absent one inherits the default styling.

Select a theme with one key:

```
theme = "academic"          // builtin name
theme = "themes/my-theme"   // local bundle
theme = false               // raw output, no theming
```

The same values work with `--theme` on `calepin compile` and `calepin watch`, and per document with `#calepin.setup(theme: ...)`. Precedence: CLI, then document, then `calepin.toml`.

To customize a builtin theme, copy it into your project and take ownership:

```
calepin new theme                    // copies the default theme to themes/
calepin/
calepin new theme --theme academic    // copies the academic theme
```

Delete any ejected file you do not intend to change; output is identical thanks to the fallback rules, and the remaining files document exactly what you own. Never edit files under `.calepin/`: that directory is regenerated on every build and edits there are lost.

Website builds pass site data into HTML themes. The bundled `calepin` theme uses this data for sidebar navigation, previous and next page links, table of contents, metadata, the top-bar brand link, and the GitHub link. Local theme templates can access:

- `site.sidebar`
- `site.sidebar_sections`
- `site.navbar_left`
- `site.navbar_center`
- `site.navbar_right`
- `site.languages`
- `site.translations`
- `site.language`
- `site.toc`
- `site.title`
- `site.description`
- `site.base_url`
- `site.logo`
- `site.logo_alt`

- `site.home_url`
- `site.github_url`
- `site.current_url`
- `site.page_title`
- `snippets.css.theme`
- `snippets.css.code`
- `snippets.css.widgets`
- `snippets.js.copy_code`
- `snippets.js.language_picker`
- `snippets.js.theme_toggle`
- `snippets.typst.code_block`

Entries in `site.sidebar`, `site.sidebar_sections`, and the navbar regions expose `href`, `label`, `label_html`, `active`, and `widget`. Link entries have `widget = none`. Widget entries preserve the configured string, so a custom theme can invent names such as `search` or `profile-menu` and render them by checking `item.widget`. The bundled themes understand `widget = "theme"` and `widget = "language"`.

Bundled snippets are available to local HTML themes through the `snippets` object. For example, include reusable base styling, widget styling, code/output styling, and widget behavior with:

```
<style>{{ snippets.css.theme }}</style>
<style>{{ snippets.css.code }}</style>
<style>{{ snippets.css.widgets }}</style>
<script>{{ snippets.js.copy_code }}</script>
<script>{{ snippets.js.language_picker }}</script>
<script>{{ snippets.js.theme_toggle }}</script>
```

`snippets.css.theme` is the shared visual base used by the bundled single-document HTML theme, website theme, and academic theme. It owns common typography, heading scale, Pico primary colors, accent variables, code/output variables, figure defaults, and global document defaults. The bundled themes keep only the layout decisions that are genuinely different, such as sidebars, top bars, table-of-contents placement, profile widgets, and the single-document view switcher.

`snippets.css.widgets` and the matching JavaScript snippets keep interactive controls consistent across themes:

- `snippets.js.theme_toggle` enhances controls marked with `data-calepin-theme-toggle`
- `snippets.js.language_picker` enhances selects marked with `data-calepin-language-picker`

Custom themes should prefer these snippets before copying bundled theme CSS or JavaScript. That keeps dark-mode controls, language pickers, code blocks, and base typography aligned with future Calepin releases.

The bundle's `paged.typ` is inserted after Calepin's executable-fence rules and before each page source for PDF, SVG, and PNG output. Calepin also stages reusable Typst snippets under `/.calepin/snippets/typst/`; the bundled `calepin` theme's `paged.typ` imports `code-block.typ` from there. A minimal `paged.typ` can replace the default source-block styling:

```
#import "/.calepin/snippets/typst/code-block.typ": code-block

#show raw.where(block: true): it => {
  if sys.inputs.at("calepin-target", default: "paged") == "html" {
    it
```

```
  } else {
    code-block(it)
  }
}
```

Source and PDF views

The bundled website theme includes a view switcher for rendered HTML, source, and PDF.

The source view is powered by a JSON script embedded in each generated HTML page. The PDF view expects the matching `.pdf` output generated by the default website build.

If you build with `--format html`, the PDF files are not generated. In that mode, the theme can still render HTML and source views, but the PDF view will not have a matching file.